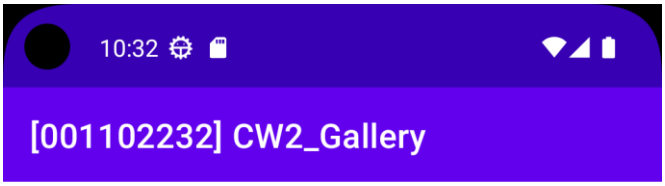
COMP1661 Logbook Upload Template

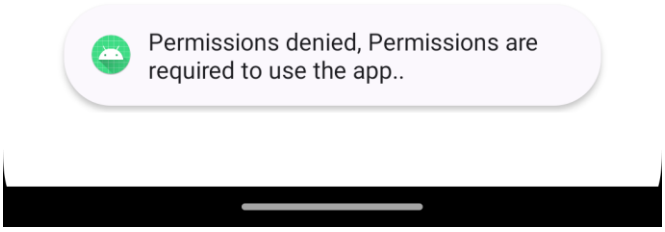
1. Basic Information

1.1 Student name	**Marzuq Sogbanmu**
1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work.	**Name: N/A** **Login id: N/A**
1.3 Which Exercise is this? Tick as appropriate.	• Exercise 1 ☐ • Exercise 2 ☒ • Exercise 3 ☐
1.4 How well did you complete the exercise? Tick as appropriate.	• I tried but couldn't complete it ☒ • I did it but I feel I should have done better ☐ • I did everything that was asked ☐ • I did more than was asked for ☐
1.5 Briefly explain your answer to question 1.4	**I was having issues with getting my application to grant permission to read external storage to get the pictures for the gallery, and so no images would be displayed. Screenshot provided**

Exercise answer

2.1 Screen shots demonstrating what you achieved





2.2 Code that you wrote

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_gravity="center"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <!--recycler view for displaying the list of images-->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/idRVImages"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</RelativeLayout>
```

Activity_image_detail.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ImageDetailActivity">

    <!--image view to display our image-->
    <ImageView
        android:id="@+id/idIVImage"
        android:layout_width="match_parent"
        android:layout_height="300dp"
        android:layout_centerInParent="true" />

</RelativeLayout>
```

Card_layout.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_margin="3dp"
    android:elevation="8dp"
    app:cardCornerRadius="8dp">

    <!--Image view for displaying the image
        in our card layout in recycler view-->
    <ImageView
        android:id="@+id/idIVImage"
        android:layout_width="100dp"
        android:layout_height="100dp"
        android:layout_gravity="center"
        android:scaleType="centerCrop" />
```

```
</androidx.cardview.widget.CardView>
```

MainActivity.java

```
package com.example.a001102232cw2_gallery;

import android.content.pm.PackageManager;
import android.database.Cursor;
import android.os.Bundle;
import android.provider.MediaStore;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.GridLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;

import static android.Manifest.permission.READ_EXTERNAL_STORAGE;

public class MainActivity extends AppCompatActivity {

    // on below line we are creating variables for
    // our array list, recycler view and adapter class.
    private static final int PERMISSION_REQUEST_CODE = 200;
    private ArrayList<String> imagePaths;
    private RecyclerView imagesRV;
    private RecyclerViewAdapter imageRVAdapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);



        // creating a new array list and
        // initializing our recycler view.
        imagePaths = new ArrayList<>();
        imagesRV = findViewById(R.id.idRVImages);

        // we are calling a method to request
        // the permissions to read external storage.
        requestPermissions();

        // calling a method to
        // prepare our recycler view.
        prepareRecyclerView();
    }

    private boolean checkPermission() {
        // in this method we are checking if the permissions are granted or not and
returning the result.
        int result = ContextCompat.checkSelfPermission(getApplicationContext(),
READ_EXTERNAL_STORAGE);
        return result == PackageManager.PERMISSION_GRANTED;
    }

    private void requestPermissions() {
        if (checkPermission()) {
            // if the permissions are already granted we are calling
            // a method to get all images from our external storage.
            Toast.makeText(this, "Permissions granted..", Toast.LENGTH_SHORT).show();
```

```
            getImagePath();
        } else {
            // if the permissions are not granted we are
            // calling a method to request permissions.
            requestPermission();
        }
    }

    private void requestPermission() {
        //on below line we are requesting the read external storage permissions.
        ActivityCompat.requestPermissions(this, new String[]{READ_EXTERNAL_STORAGE},
PERMISSION_REQUEST_CODE);
    }

    private void prepareRecyclerView() {

        // in this method we are preparing our recycler view.
        // on below line we are initializing our adapter class.
        imageRVAdapter = new RecyclerViewAdapter(MainActivity.this, imagePaths);

        // on below line we are creating a new grid layout manager.
        GridLayoutManager manager = new GridLayoutManager(MainActivity.this, 4);

        // on below line we are setting layout
        // manager and adapter to our recycler view.
        imagesRV.setLayoutManager(manager);
        imagesRV.setAdapter(imageRVAdapter);
    }

    private void getImagePath() {
        // in this method we are adding all our image paths
        // in our arraylist which we have created.
        // on below line we are checking if the device is having an sd card or not.
        boolean isSDPresent =
android.os.Environment.getExternalStorageState().equals(android.os.Environment.MEDIA_MO
UNTED);

        if (isSDPresent) {

            // if the sd card is present we are creating a new list in
            // which we are getting our images data with their ids.
            final String[] columns = {MediaStore.Images.Media.DATA,
MediaStore.Images.Media._ID};

            // on below line we are creating a new
            // string to order our images by string.
            final String orderBy = MediaStore.Images.Media._ID;

            // this method will stores all the images
            // from the gallery in Cursor
            Cursor cursor =
getContentResolver().query(MediaStore.Images.Media.EXTERNAL_CONTENT_URI, columns, null,
null, orderBy);

            // below line is to get total number of images
            int count = cursor.getCount();

            // on below line we are running a loop to add
            // the image file path in our array list.
            for (int i = 0; i < count; i++) {

                // on below line we are moving our cursor position
                cursor.moveToPosition(i);

                // on below line we are getting image file path
                int dataColumnIndex =
cursor.getColumnIndex(MediaStore.Images.Media.DATA);
```

```
                // after that we are getting the image file path
                // and adding that path in our array list.
                imagePaths.add(cursor.getString(dataColumnIndex));
            }
            imageRVAdapter.notifyDataSetChanged();
            // after adding the data to our
            // array list we are closing our cursor.
            cursor.close();
        }
    }

    @Override
    public void onRequestPermissionsResult(int requestCode, String permissions[], int[]
grantResults) {
        // this method is called after permissions has been granted.
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        switch (requestCode) {
            // we are checking the permission code.
            case PERMISSION_REQUEST_CODE:
                // in this case we are checking if the permissions are accepted or not.
                if (grantResults.length > 0) {
                    boolean storageAccepted = grantResults[0] ==
PackageManager.PERMISSION_GRANTED;
                    if (storageAccepted) {
                        // if the permissions are accepted we are displaying a toast
message
                        // and calling a method to get image path.
                        Toast.makeText(this, "Permissions Granted..",
Toast.LENGTH_SHORT).show();
                        getImagePath();
                    } else {
                        // if permissions are denied we are closing the app and
displaying the toast message.
                        Toast.makeText(this, "Permissions denied, Permissions are
required to use the app..", Toast.LENGTH_SHORT).show();
                    }
                }
                break;
        }
    }
}
```

RecyclerViewAdapter.java

```
package com.example.a001102232cw2_gallery;

import android.content.Context;
import android.content.Intent;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.squareup.picasso.Picasso;

import java.io.File;
import java.util.ArrayList;

public class RecyclerViewAdapter extends
RecyclerView.Adapter<RecyclerViewAdapter.RecyclerViewHolder> {

    // creating a variable for our context and array list.
```

```
    private final Context context;
    private final ArrayList<String> imagePathArrayList;

    // on below line we have created a constructor.
    public RecyclerViewAdapter(Context context, ArrayList<String> imagePathArrayList) {
        this.context = context;
        this.imagePathArrayList = imagePathArrayList;
    }

    @NonNull
    @Override
    public RecyclerViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        // Inflate Layout in this method which we have created.
        View view =
LayoutInflater.from(parent.getContext()).inflate(R.layout.card_layout, parent, false);
        return new RecyclerViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull RecyclerViewHolder holder, int position) {

        // on below line we are getting the file from the
        // path which we have stored in our list.
        File imgFile = new File(imagePathArrayList.get(position));

        // on below line we are checking if the file exists or not.
        if (imgFile.exists()) {

            // if the file exists then we are displaying that file in our image view
using picasso library.

Picasso.get().load(imgFile).placeholder(R.drawable.ic_launcher_background).into(holder.
imageIV);

            // on below line we are adding click listener to our item of recycler view.
            holder.itemView.setOnClickListener(new View.OnClickListener() {
                @Override
                public void onClick(View v) {

                    // inside on click listener we are creating a new intent
                    Intent i = new Intent(context, ImageDetailActivity.class);

                    // on below line we are passing the image path to our new activity.
                    i.putExtra("imgPath", imagePathArrayList.get(position));

                    // at last we are starting our activity.
                    context.startActivity(i);
                }
            });
        }
    }

    @Override
    public int getItemCount() {
        // this method returns
        // the size of recyclerview
        return imagePathArrayList.size();
    }

    // View Holder Class to handle Recycler View.
    public static class RecyclerViewHolder extends RecyclerView.ViewHolder {

        // creating variables for our views.
        private final ImageView imageIV;

        public RecyclerViewHolder(@NonNull View itemView) {
```

```
            super(itemView);
            // initializing our views with their ids.
            imageIV = itemView.findViewById(R.id.idIVImage);
        }
    }
}
```

ImageDetailActivity.java

```
package com.example.a001102232cw2_gallery;

import android.os.Bundle;
import android.view.MotionEvent;
import android.view.ScaleGestureDetector;
import android.widget.ImageView;

import androidx.appcompat.app.AppCompatActivity;

import com.squareup.picasso.Picasso;

import java.io.File;

public class ImageDetailActivity extends AppCompatActivity {

    // creating a string variable, image view variable
    // and a variable for our scale gesture detector class.
    String imgPath;
    private ImageView imageView;
    private ScaleGestureDetector scaleGestureDetector;

    // on below line we are defining our scale factor.
    private float mScaleFactor = 1.0f;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_image_detail);

        // on below line getting data which we have passed from our adapter class.
        imgPath = getIntent().getStringExtra("imgPath");

        // initializing our image view.
        imageView = findViewById(R.id.idIVImage);

        // on below line we are initializing our scale gesture detector for zoom in and
out for our image.
        scaleGestureDetector = new ScaleGestureDetector(this, new ScaleListener());

        // on below line we are getting our image file from its path.
        File imgFile = new File(imgPath);

        // if the file exists then we are loading that image in our image view.
        if (imgFile.exists()) {

Picasso.get().load(imgFile).placeholder(R.drawable.ic_launcher_background).into(imageVi
ew);
        }
    }

    @Override
    public boolean onTouchEvent(MotionEvent motionEvent) {
        // inside on touch event method we are calling on
        // touch event method and passing our motion event to it.
        scaleGestureDetector.onTouchEvent(motionEvent);
        return true;
    }
```

```
    private class ScaleListener extends
ScaleGestureDetector.SimpleOnScaleGestureListener {
        // on below line we are creating a class for our scale
        // listener and extending it with gesture listener.
        @Override
        public boolean onScale(ScaleGestureDetector scaleGestureDetector) {

            // inside on scale method we are setting scale
            // for our image in our image view.
            mScaleFactor *= scaleGestureDetector.getScaleFactor();
            mScaleFactor = Math.max(0.1f, Math.min(mScaleFactor, 10.0f));

            // on below line we are setting
            // scale x and scale y to our image view.
            imageView.setScaleX(mScaleFactor);
            imageView.setScaleY(mScaleFactor);
            return true;
        }
    }
}
```

This was added to build:gradle (app)

```
implementation 'com.squareup.picasso:picasso:2.71828'
```